

Technische Hochschule Ingolstadt

Fakultät Elektro- und Informationstechnik

Sensor Network Technologies and Sensor Fusion

Recap

Jan 15, 2025

Welcome to the lecture!

Prof. Dr.-Ing. Björn Giesler

Technische Hochschule Ingolstadt

AIMotion, AI and Sensor Data Fusion

Email: bjoern.giesler@thi.de

Sensor Networks and Sensor Fusion

Organizational Matters

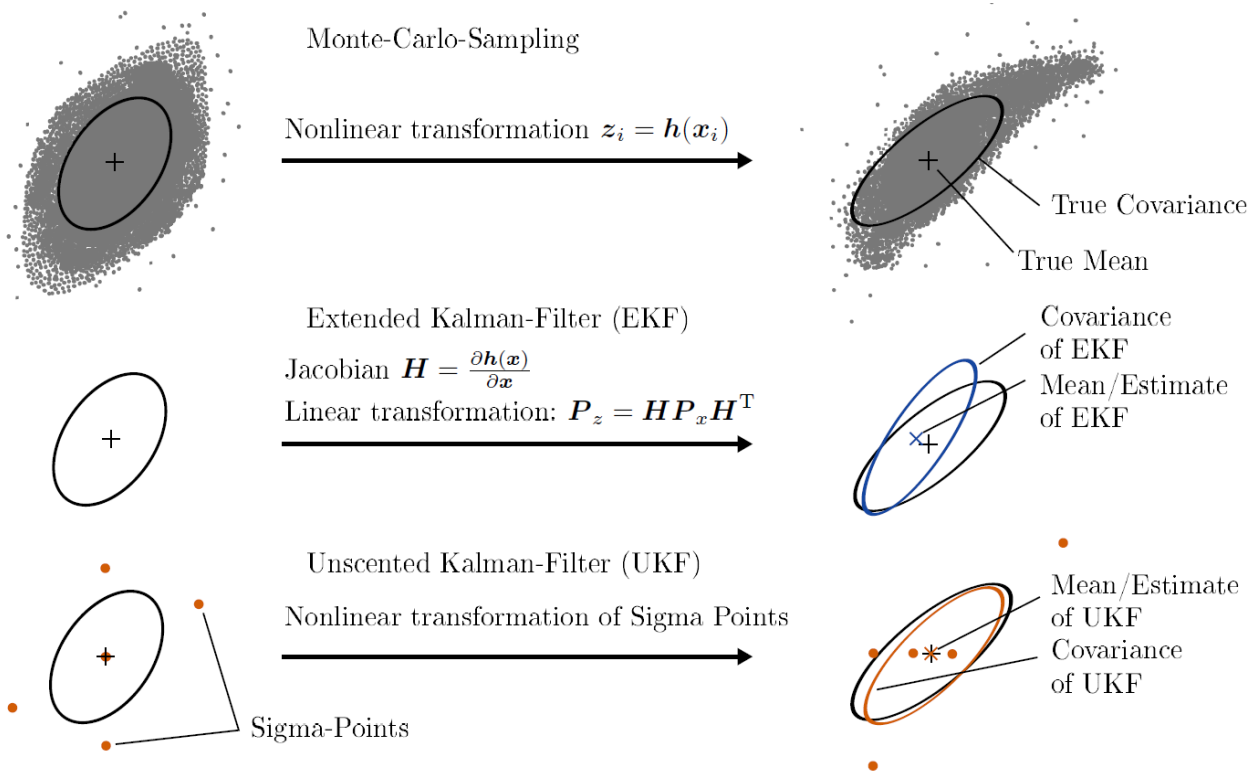
- **Please register for the lecture on Moodle!**
  - Link: https://moodle.thi.de/course/view.php?id=10580
  - Key: **SensNT+DF#WS25**
- **News on Moodle:**
  - Slides for the lecture (upcoming lectures marked as WIP and are subject to change!)
  - Formulary, Exercises (Solutions provided after discussion in the lecture)
  - Exercises are optional, one task / sheet per week, but are highly recommended.
    They can be handed in for correction.
  - I will *try* to publish exercises before Wednesday‘s lecture so that initial questions can be discussed on Thursday.
- **Exam: Written exam, 90 minutes**
- **Consultation hours: Please arrange via e-mail to bjoern.giesler@thi.de.**
- **Please also don‘t hesitate to send questions, lecture feedback, change requests, etc.!**

Let's talk exam!

- **I want to know what's in your brain! My priority is always**
  1. have you understood the concept?
  2. ...thoroughly enough so you could implement it practically?
  3. and last, are the numerical results you obtain correct?
- **No formulary, personal notes, tablets, laptops, AI assistants or personal humanoid messenger or telepresence robots allowed.**
- **Pocket calculators are a necessity.**
- **There will likely be some hand calculation with the help of pocket calculators.**
- **No coding of course, but questions about algorithms in pseudo code are possible.**
- **Long answer questions possible. Multiple choice questions possible. Formula questions possible.**
- **Basically everything we discussed in the lecture is relevant. Do not rely on only the exercise sheets or programming exercises to prepare.**
- **I will be fair!**

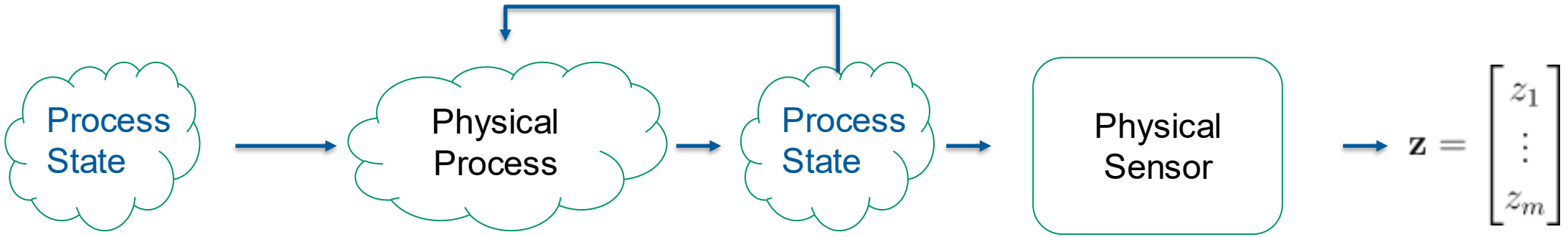
Recap KF

Recap

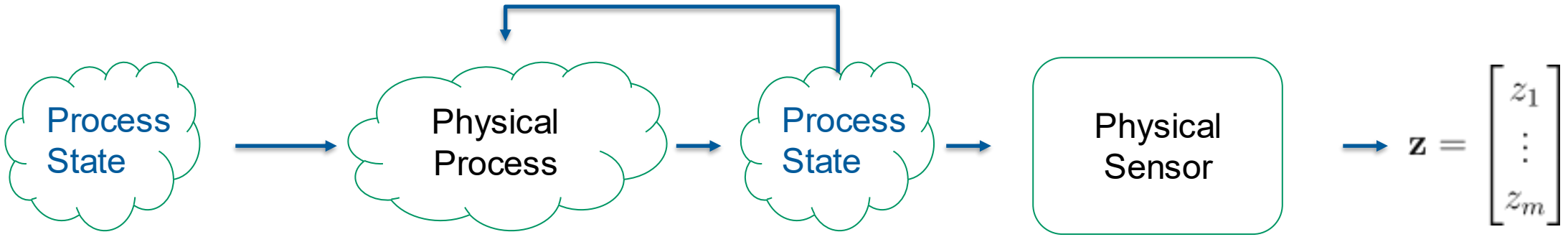


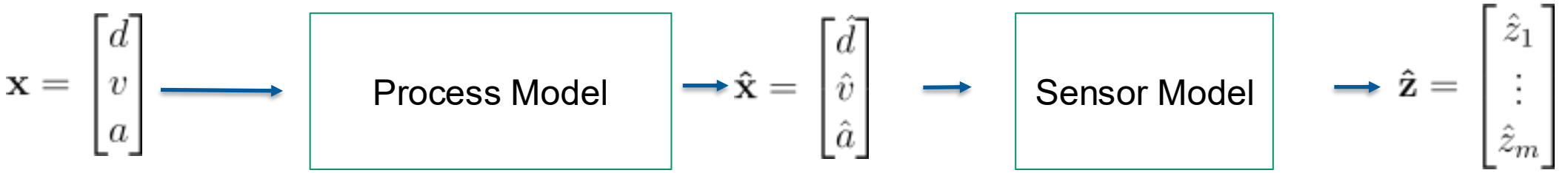
- **Even for UKF, we need to be able to at least sample the state transition function (even if we can't formulate it directly).**
- **What do we do if we don't have it available?**

Approximating State Transition Functions



Approximating State Transition Functions

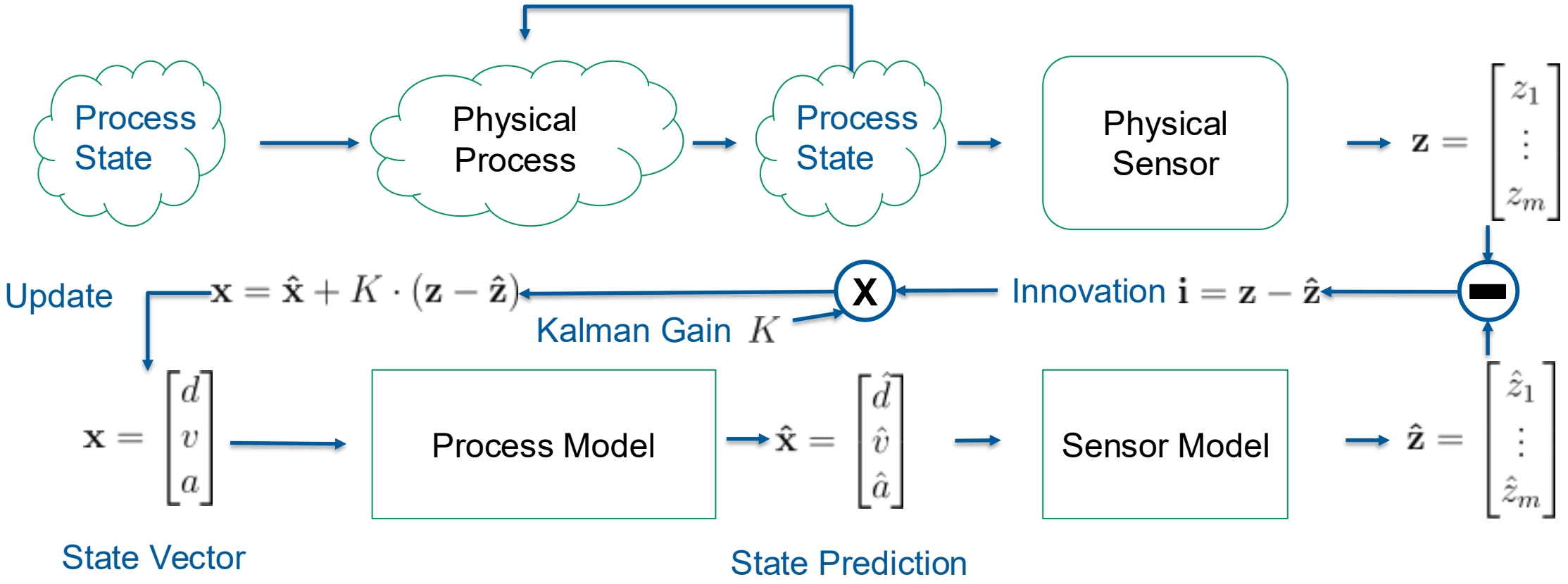




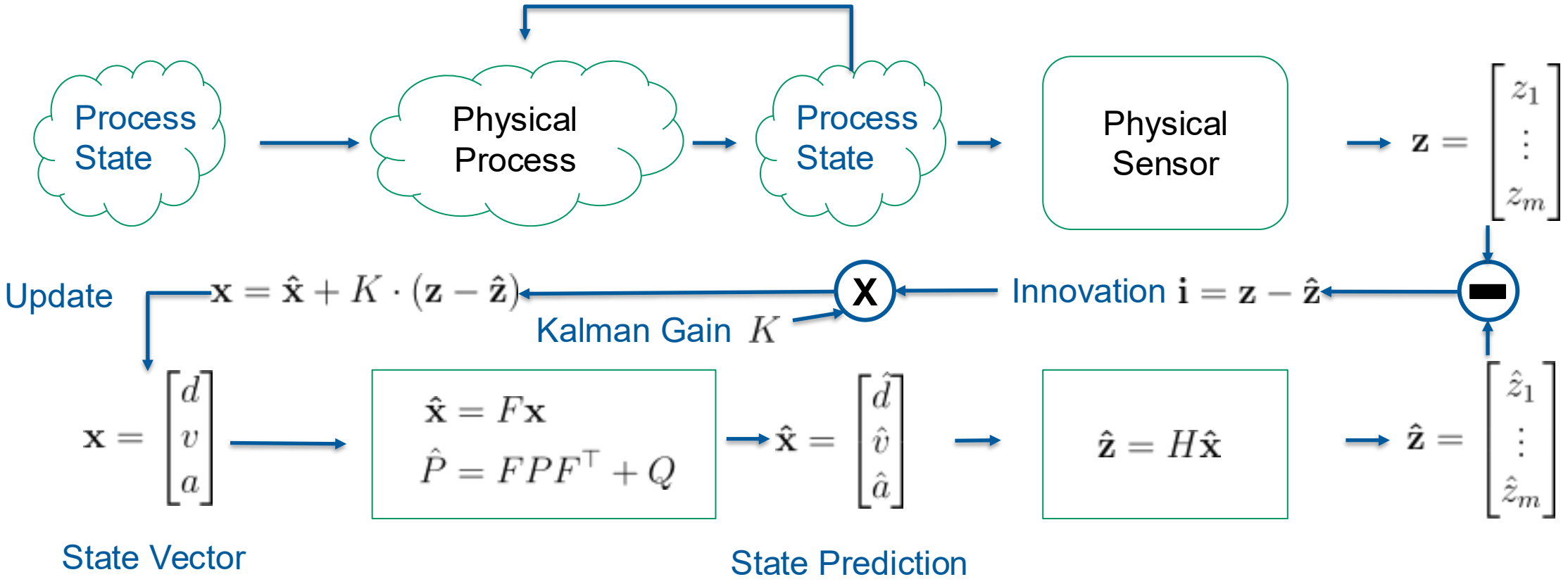
State Vector

State Prediction

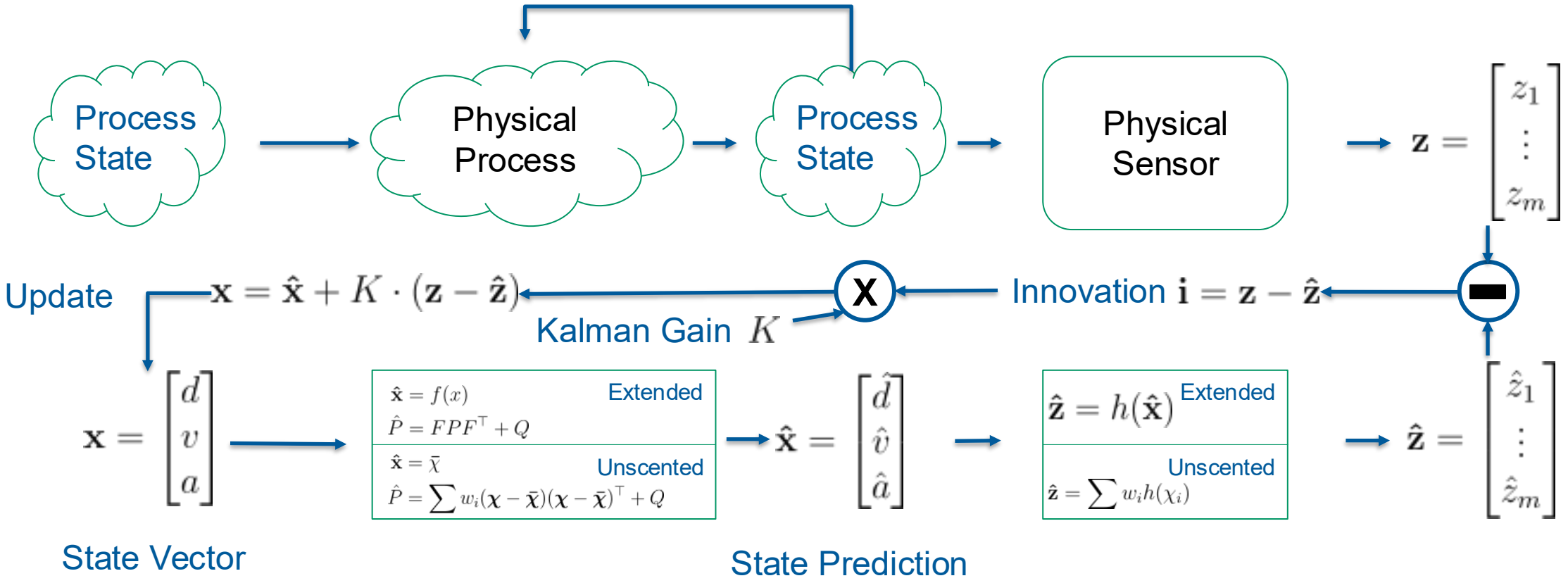
Approximating State Transition Functions



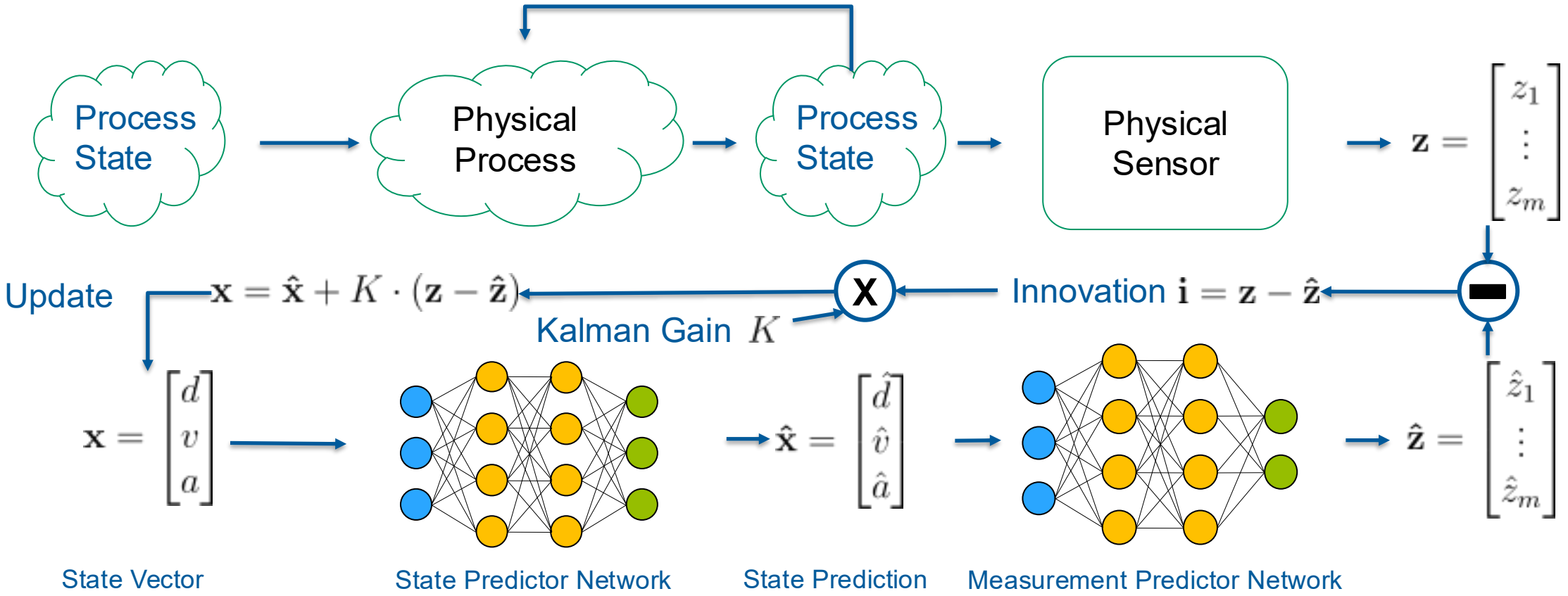
Approximating State Transition Functions



Approximating State Transition Functions



Approximating State Transition Functions with Neural Networks



Interrelating Kalman, Extended Kalman, and Unscented Kalman Filters

	Input Error Distribution	Function Transform	Output Error Distribution
Kalman Filter	Approximate	Exact (linear)	Approximate
Extended Kalman	Approximate	Approximate	Approximate
Unscented Kalman	Approximate	Exact (nonlinear)	Approximate

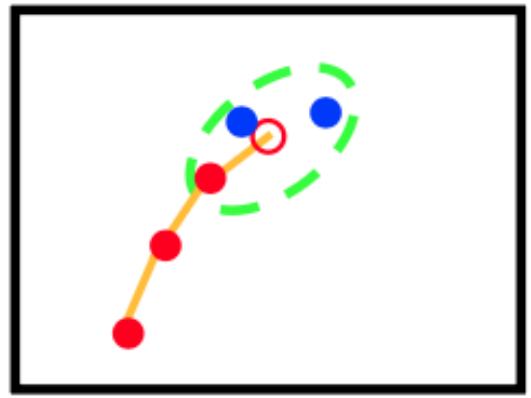
UKF Advantage:

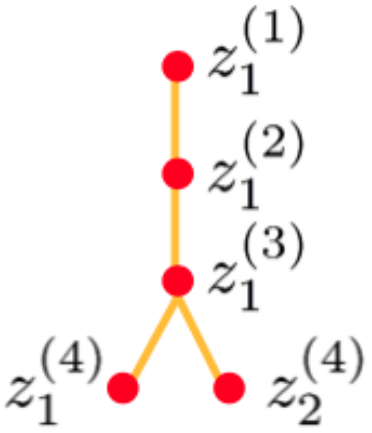
1. Nonlinear motion and sensor models do not need to be known.
2. No need to compute Jacobians.
3. Motion and sensor models need not be continuous or differentiable.
4. Computational complexity is similar to EKF.
5. Unscented Transform is accurate up to the 3rd moment for Gaussians.

Recap Multi Object Tracking

Track Splitting Filter

- **The Nearest-Neighbor filter still has uncertainties and options that can't really be resolved**
  - Uncertain decision making
  - ...through gating decision making functions
- **Rather than arbitrarily assigning the "closest" (by whatever norm) measurement, form a tree.**
- **Branches denote alternate assignments. No *decision* is made at this stage; decisions are postponed until additional measurements have been gathered.**
- **Potential problem:**
  Track trees can quickly become very large due to combinatorial explosion.
- **We need some measure of the likelihood so we can prune the tree.**





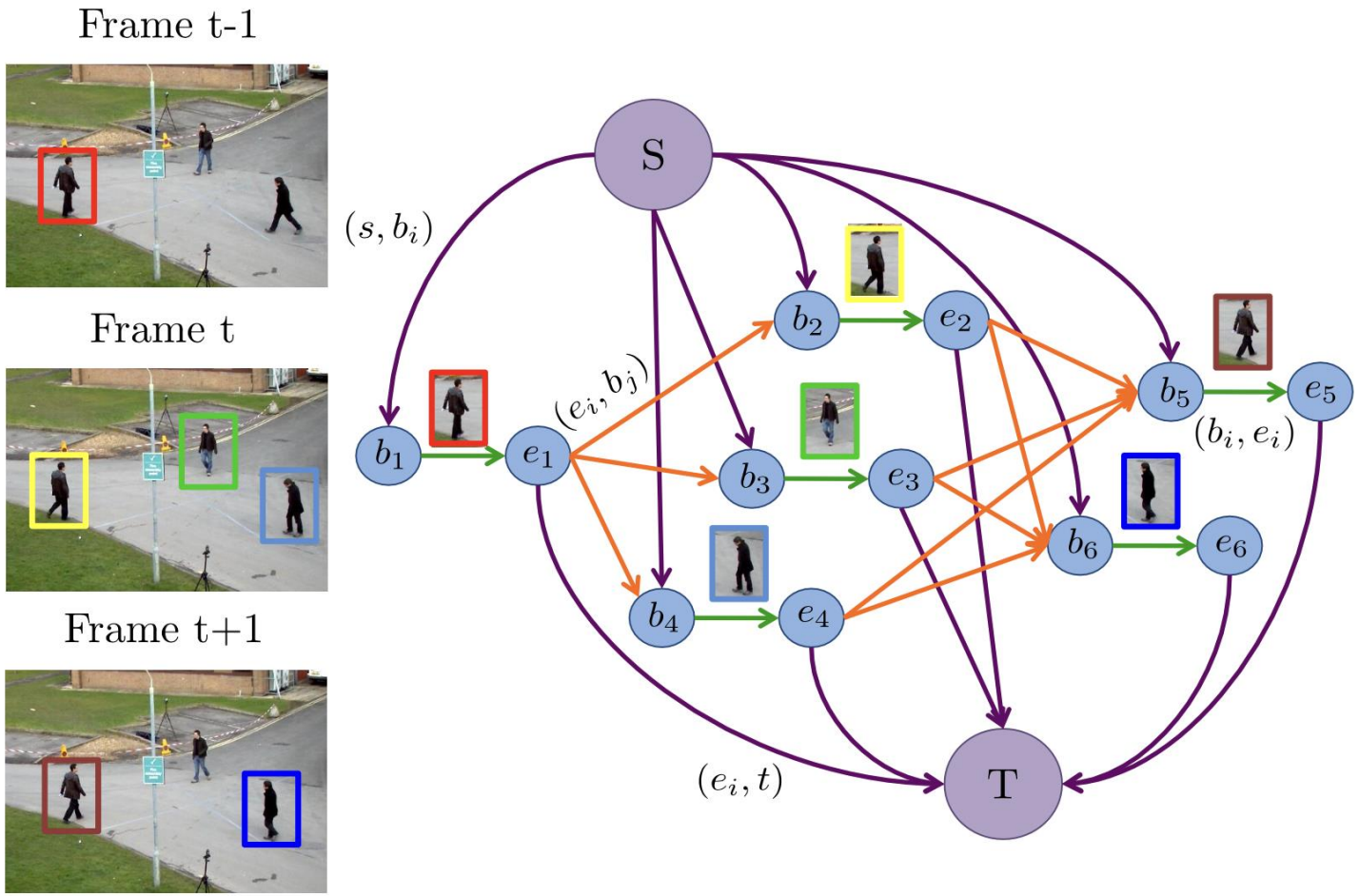
Matching with similarity assignments

- **Form a matrix of pairwise similarity scores.**
- **Similarity could be**
  - based on motion prediction (e.g. Mahalanobis Distance)
  - based on appearance
  - combination
- **Goal:**
  - Choose one match from each row and column to maximize the sum of scores

Frame $t+1$

Frame t			
	0.11	**0.95**	0.23
	0.85	0.25	**0.89**
	**0.90**	0.12	0.81

Tracking over multiple frames by network flow optimization



Recap Particle Filter

Recap Particle Filter

- **Kalman filters assume a *unimodal normal distribution* that can be modeled using mean and variance.**
- **They cannot deal with multimodal (multi-peak) distributions at all, and are not very good at non-normal distributed error.**
- **Multi Object Tracking solves this for the specific problem of measurement-to-object assignment**
- **Real world problems often have no clear local or even global optimum at one state in the observation timeline.**
- **Particle filters work by**
  - inherently representing the problem with multiple hypotheses
  - weighing the hypotheses according to the likelihood of the current observation under the precondition of the hypothesis parameters
  - resampling using the likelihood weights so that more hypotheses get created around the likeliest current states.



1. Sample from prior belief $q(\mathbf{x})$ (e.g. the uniform distribution)

distribution)



2. Compute importance weights, $w(\mathbf{x}) = p(\mathbf{x})/q(\mathbf{x})$



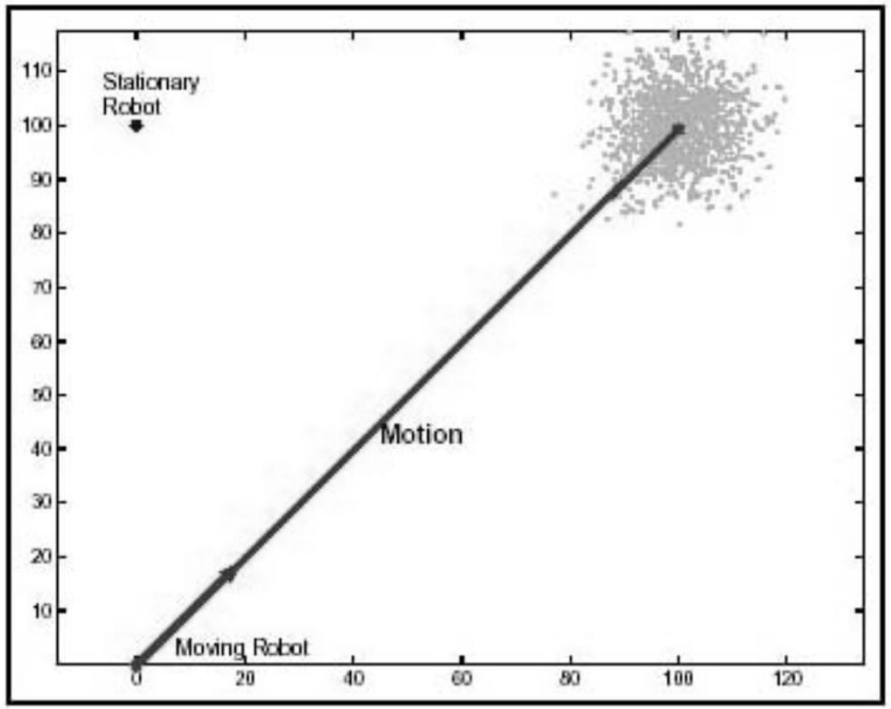
3. Resample particles according to importance weights to get $p(\mathbf{x})$.
Samples with high weights chosen many times, density reflects PDF.

1. Prediction

- **Prediction: For each particle, sample and add random, noisy values from action model.**

- **Resulting proposal distribution**

$$q(\mathbf{x}) \approx p(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{u}_{t-1})p(\mathbf{x}_{t-1}|d_{0\ldots t-1})d\mathbf{x}_{t-1}$$



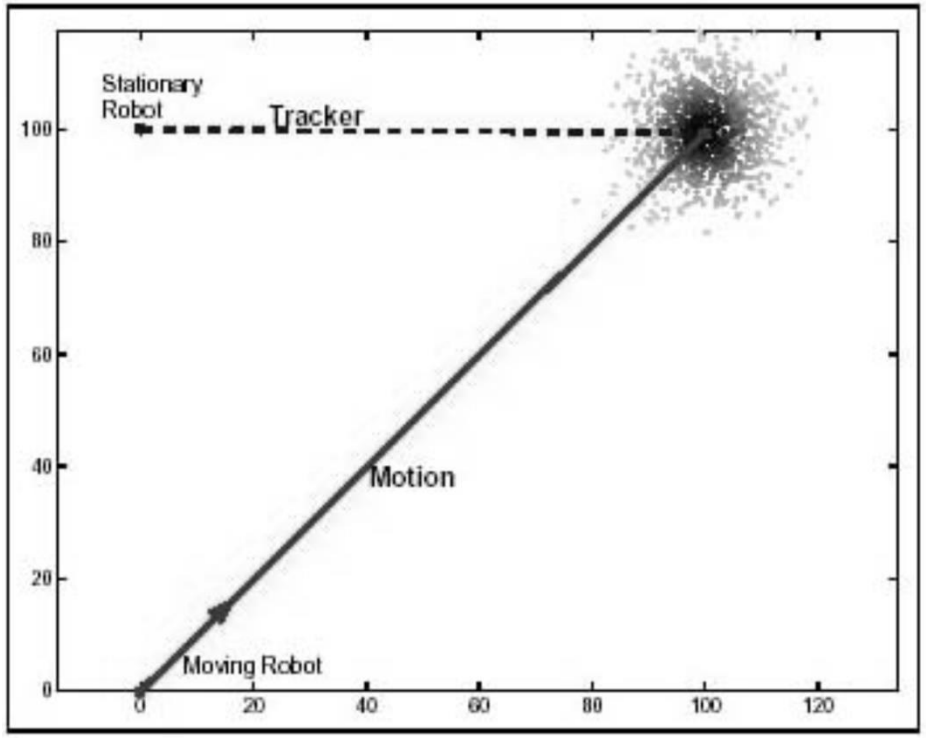
2. Update

- Each particle's weight is the likelihood of getting the sensor readings from that particle's hypothesis.

- The weight associated with each particle is

$$w(\mathbf{x}) = p(\mathbf{x})/q(\mathbf{x}) = p(\mathbf{z}_t|\mathbf{x}_t)$$

normalized so that all the weights sum to 1.



3. Resample

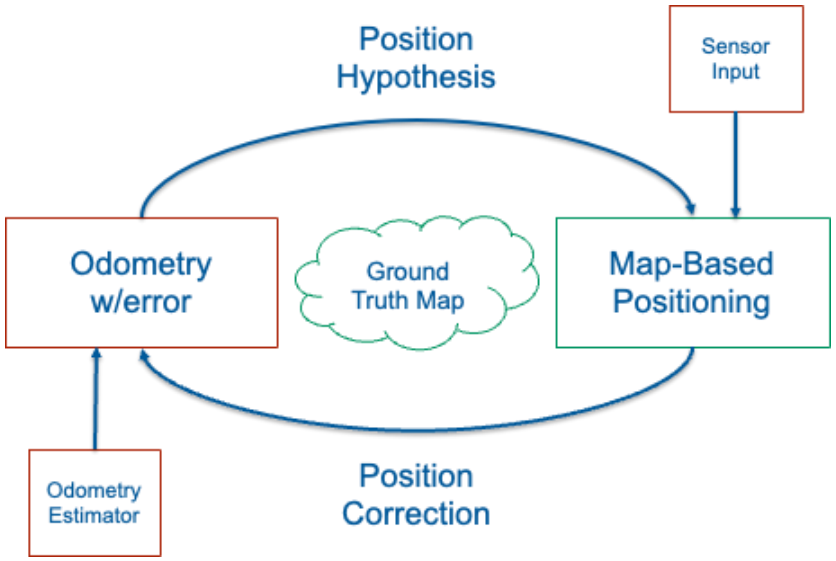
- New set of particles are chosen such that each particle survives in proportion to its weight.



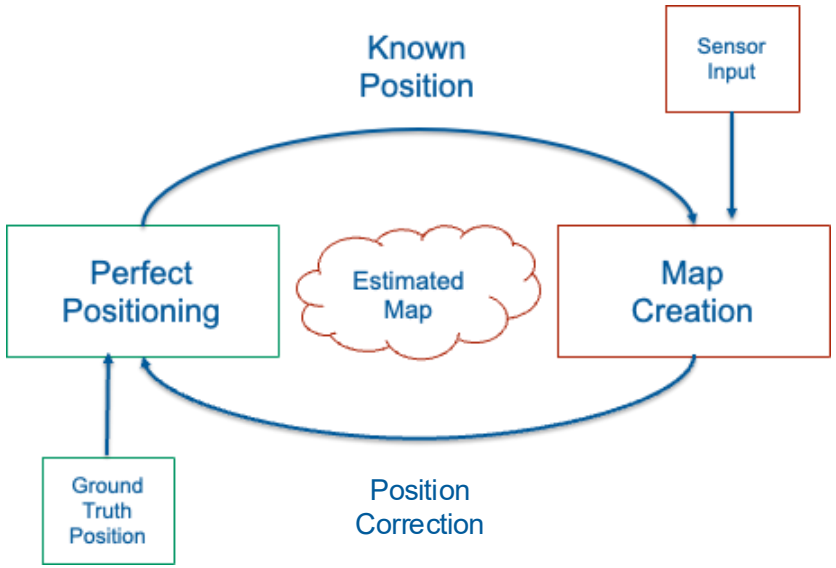
- Resulting distribution is

$$\underbrace{p(\mathbf{x}_t|d_{0\ldots t})}_{\text{Posterior}} = \underbrace{\eta \cdot p(\mathbf{z}_t|\mathbf{x}_t)}_{\text{Weights}} \int \underbrace{p(\mathbf{x}_t|\mathbf{u}_{t-1},\mathbf{x}_{t-1})p(\mathbf{x}_{t-1}|d_{0\ldots t-1})d\mathbf{x}_{t-1}}_{\text{Prior}}$$

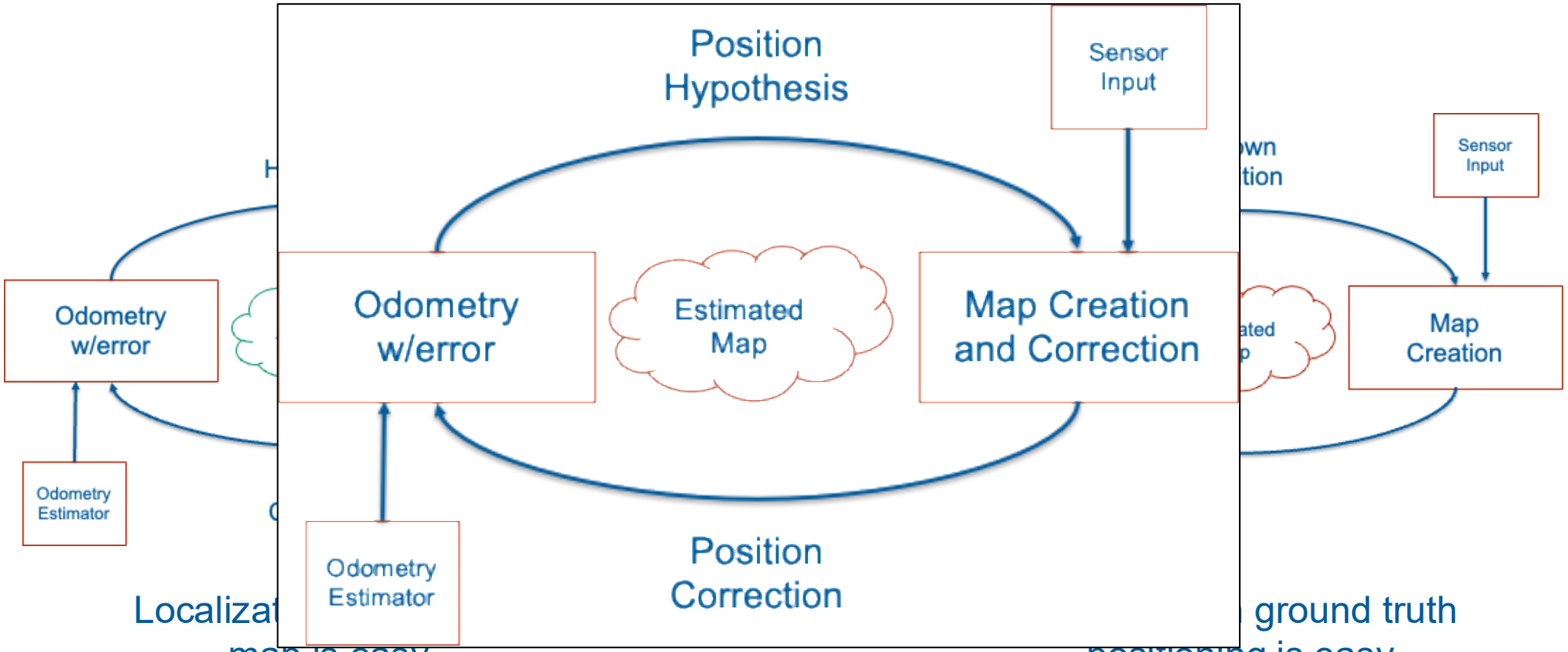
Recap SLAM



Localization in a known map is easy



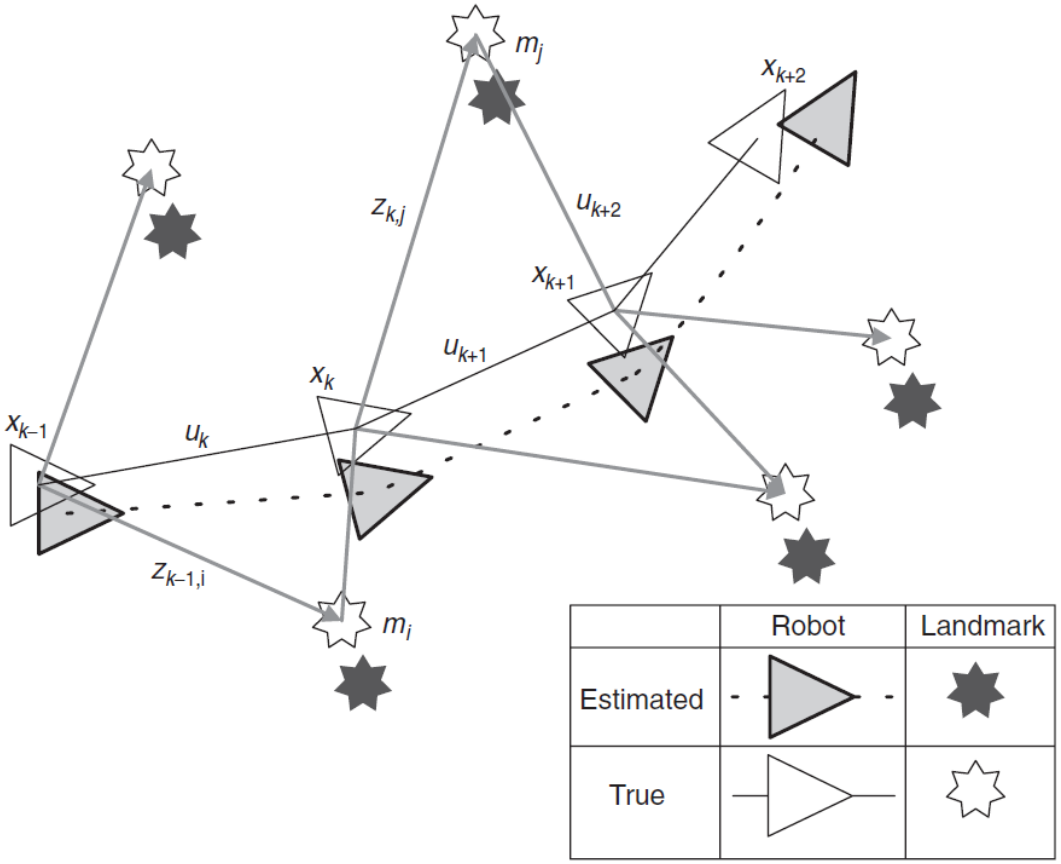
Mapping with ground truth positioning is easy



Localizat ground truth
map is easy positioning is easy

SLAM is the technique of estimating
position and map at the same time.

- **Robot pose and landmar positions are highly inter-correlated**
- **Two basic ways to solve SLAM problem:**
  - *Integrate the correlation:*
    Estimate the whole system state in one Kalman Filter
    One robot pose, one map
  - *Separate the correlation:*
    Estimate the map assuming perfect positioning
    Solve the robot pose uncertainty with many hypotheses ("particles")
    Determine the currently best hypothesis by particle filter weighing / sampling
    Many hypotheses for the robot pose, one map for each



EKF SLAM

- Estimate robot's pose and locations of landmarks in the environment with one filter
- State space (for a robot moving in 2D) is

$$\mu_t = (\underbrace{x_t, y_t, \theta_t}_{\text{robot pose } \mathbf{x}_t}, \underbrace{m_{1,x}, m_{1,y}}_{\text{landmark } \mathbf{m}_1}, \ldots, \underbrace{m_{n,x}, m_{n,y}}_{\text{landmark } \mathbf{m}_n})^\top$$

- Covariance matrix models joint uncertainties between robot pose and individual landmarks:

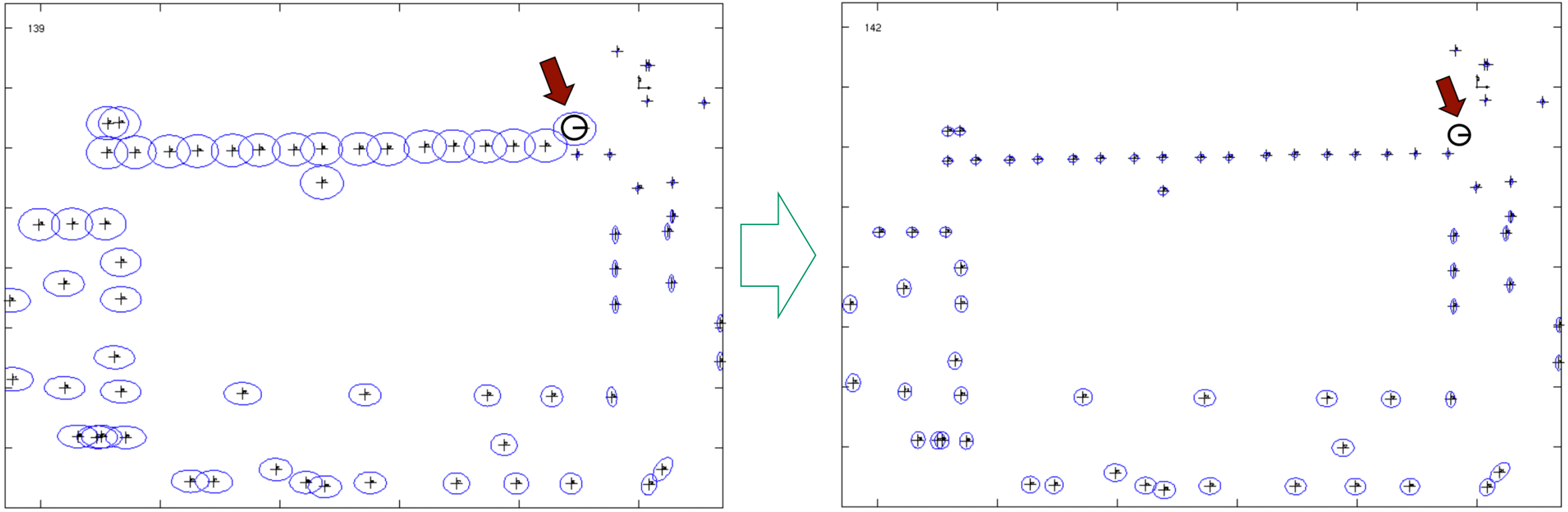
$$\Sigma = \begin{bmatrix} \Sigma_{\mathbf{x}_t\mathbf{x}_t} & \Sigma_{\mathbf{x}_t\mathbf{m}_1} & \cdots & \Sigma_{\mathbf{x}_t\mathbf{m}_n} \\ \Sigma_{\mathbf{m}_1\mathbf{x}_t} & \Sigma_{\mathbf{m}_1\mathbf{m}_1} & \cdots & \Sigma_{\mathbf{m}_1\mathbf{m}_n} \\ \vdots & \vdots & \ddots & \vdots \\ \Sigma_{\mathbf{m}_n\mathbf{x}_t} & \Sigma_{\mathbf{m}_n\mathbf{m}_1} & \cdots & \Sigma_{\mathbf{m}_n\mathbf{m}_n} \end{bmatrix}$$

or more compact:

$$\mathbf{s} = \begin{bmatrix} \mathbf{x} \\ \mathbf{m} \end{bmatrix}$$

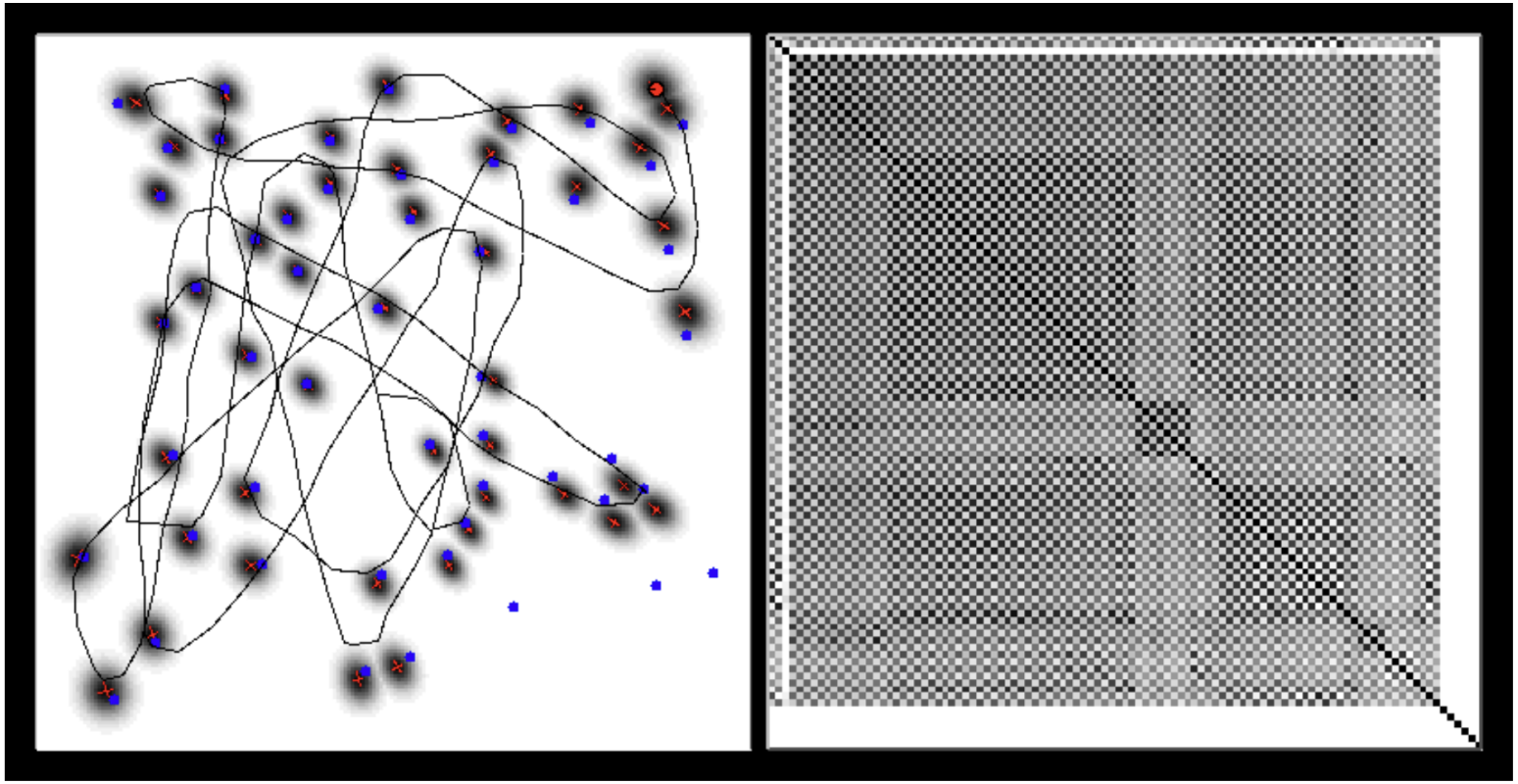
$$\Sigma = \begin{bmatrix} \Sigma_{\mathbf{xx}} & \Sigma_{\mathbf{xm}} \\ \Sigma_{\mathbf{mx}} & \Sigma_{\mathbf{mm}} \end{bmatrix}$$

Loop Closing



EKF-SLAM Correlations

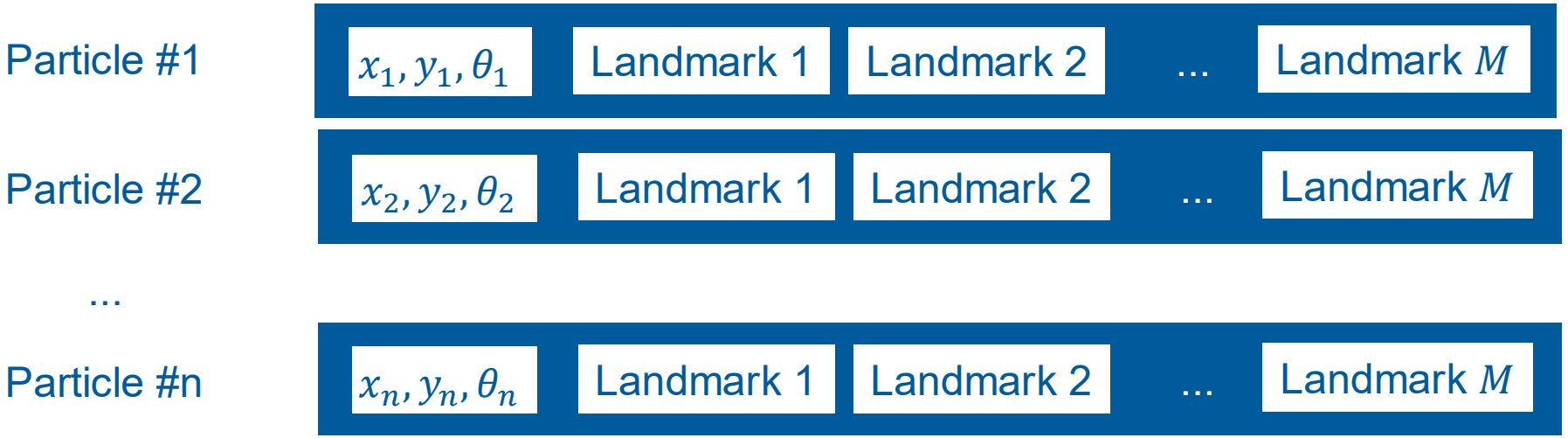
- Correlations can be observed by the state of the correlation matrix Σ:



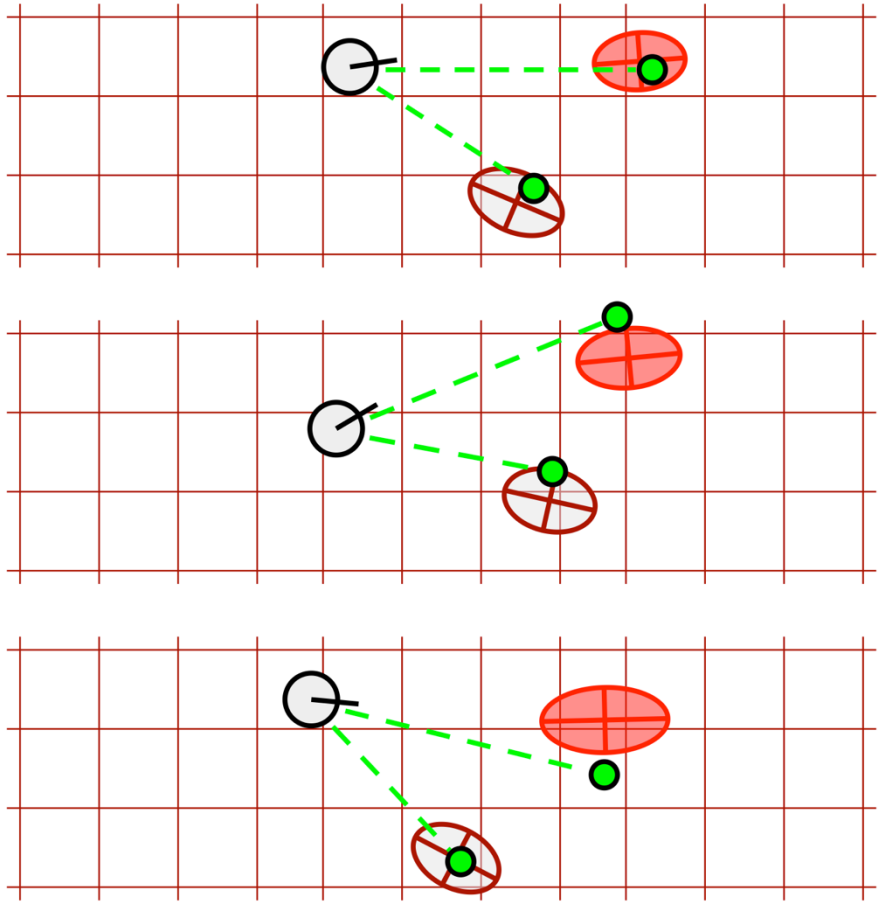
Map | Correlation Matrix

Particle Filter SLAM (FastSLAM + derivatives)

- **FastSLAM [2] is a SLAM algorithm using Rao-Blackwellized Particle Filter / Kalman Filter combination.**
  1. estimate the robot pose using a particle filter (solving the localization problem),
  2. associate each particle with a map represented by a set of Gaussian-distributed landmark positions (solving the mapping problem),
  3. estimate the landmark positions using separate EKFs,
  4. compute particle weights based on correspondence between landmarks and observations (e.g. with Mahalanobis).



FastSLAM – Map Update



Particle weight based on observation / landmark correspondence

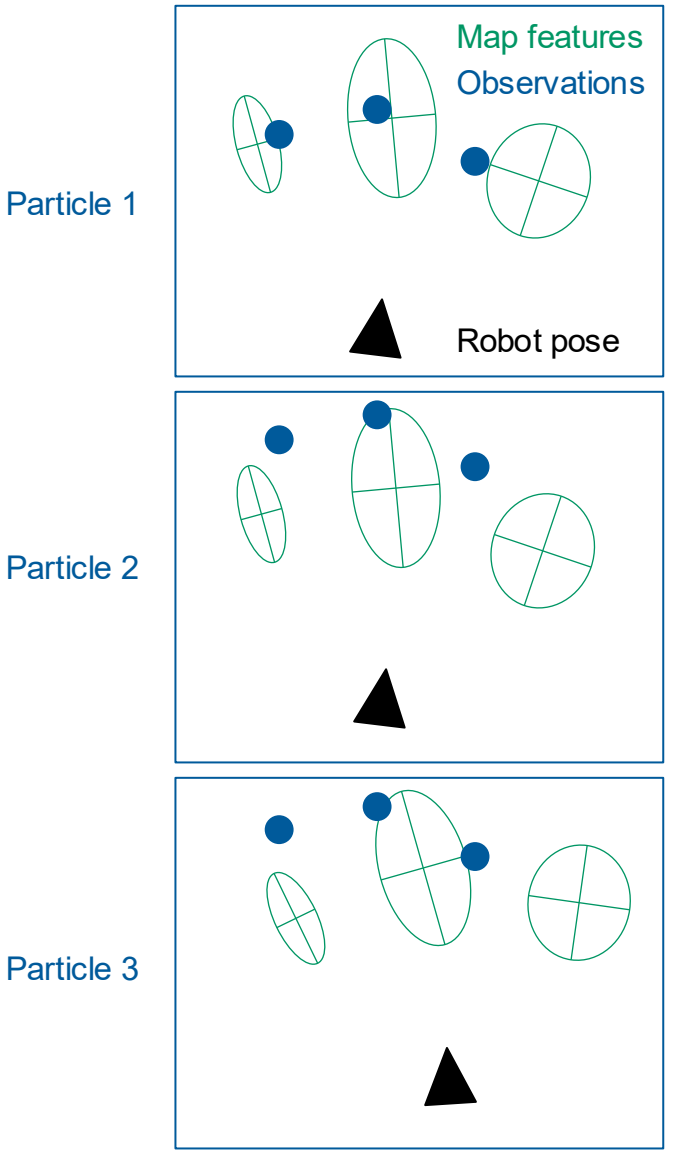
Weight = 0.6

Weight = 0.3

Weight = 0.1

FastSLAM 1.0 → 2.0

- **FastSLAM 2.0 samples from** $x_t^{[k]} \sim p(x_t | x_{1:t-1}^{[k]}, u_{1:t}, z_{1:t}).$
- **I.e., it takes into account the sensor data.**
- **For example,**
  - multiply weights with average Mahalanobis distance of map features to observations (*matching criterion)*
  - use least-squares method to optimize position parameters based on map/observation matches, then weigh based on some distance measure of the particle to that "optimal" position
  - ...
- **This results in a more peaked proposal distribution.**
- **Less particles required, more robust and accurate**
- **But more complex...**





Technische Hochschule Ingolstadt

Fakultät Elektro- und Informationstechnik

Thank you very much!